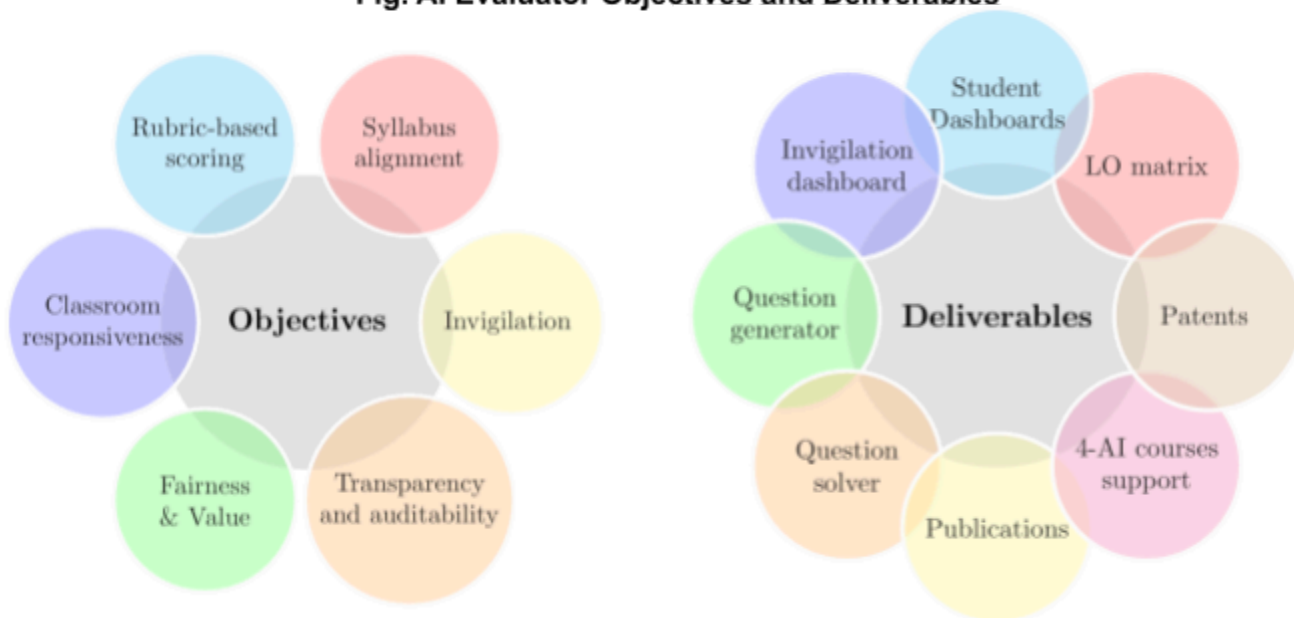


AI Evaluator

AI Evaluator is a small, dependable assistant for short, structured viva exams and exam copies corrections. This assistant will design every question and solutions based on the approved syllabus and the course learning outcomes (LOs), so teachers can trace where it came from and why it was asked. The emphasis is on Bloom's Level-4—analysis—so students are pushed to examine, compare, and explain, not just recall facts.

Fig. AI Evaluator Objectives and Deliverables



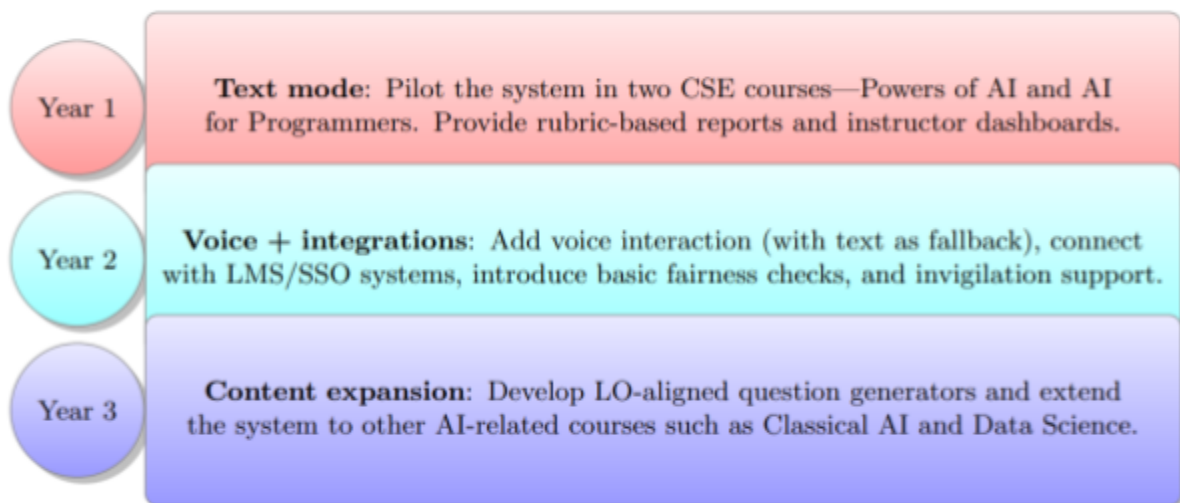
What the assistant actually does

- **Keeps questions aligned with scope:** Every prompt is tied to a specific learning outcome (LO). Duplicate or near-duplicate questions are filtered out so that students don't face repetition.
- **Uses an analytic rubric for scoring:** Each response is evaluated on key dimensions such as accuracy, clarity, and completeness. Scores come with a short explanation (1–2 lines) to make the reasoning behind them transparent.
- **Delivers useful reports:** Students get focused, easy-to-understand feedback, while instructors receive summary dashboards showing score ranges, trends, and common strengths or gaps in understanding.
- **Preserves a full record:** Transcripts, rubric versions, and notes from scoring are archived to support review, moderation, and any appeals.

Team at a glance

A faculty lead from IIT BHU, two JRFs, an industry mentor is the core team. We also have a small rubric/evaluation group, and student TA assistants share the work.

Fig. 1: Overall Plan for AI Evaluator



Why do the courses use an AI-based Evaluator?

Limitations of the current viva system

Oral examinations such as vivas and short presentations are important because they allow instructors to probe how well a student has actually understood a topic. They also give room for immediate follow-up questions whenever gaps or misconceptions appear. However, when the same approach is extended to large classes, several difficulties quickly emerge:

1. **Heavy workload on faculty:** Running individual viva sessions for a large number of students takes a lot of time and effort, and maintaining the same level of consistency throughout becomes a challenge.
2. **Differences across examiners:** Since each faculty member may frame questions in their own way, the level of difficulty and the scoring standards are not always aligned.
3. **Limited feedback for students:** In many cases, students only get a numerical score, without any explanation or guidance that could help them learn from the assessment.
4. **Weak record-keeping:** When transcripts or scoring notes are missing, it becomes difficult to explain grading decisions or handle appeals in a transparent way.

How the AI Evaluator addresses these issues (focused on Bloom's Level-4)

The evaluator is not intended to replace teachers; rather, it is meant to support them in areas where large class sizes, fairness, and consistency are especially important. Its design addresses several common challenges:

- **Curriculum alignment:** Each question is tied to an approved learning outcome (LO). This keeps the assessment within scope and helps avoid repetition.
- **Emphasis on reasoning over recall:** Prompts are framed to encourage students to compare alternatives, explain relationships, and justify their thinking—skills that match Bloom's Level-4.

- **Rubric-based scoring with explanations:** Responses are judged against clear criteria, with a short note (one or two lines) attached to explain why a particular score was given.
- **Structured reporting:** Students get a concise feedback report, while instructors receive summaries showing overall trends, recurring mistakes, and exceptional cases.
- **Audit-ready records:** The system stores transcripts, score explanations, and rubric versions so that reviews or rechecks can be conducted fairly and transparently.

3. Review of the Literature

The proposed AI-assistive reading app is mainly an integration of OCR and TTS functionalities. We now describe various related works as follows.

- "Automatic Reader" [1] utilizes Tesseract for OCR and open-source E-speak TTS for text-to-speech.
- "SPAVIS Mobile App" [2] leverages Flutter edge detection and Flutter TTS to achieve a faster API response times; however it lacks multilingual capabilities.
- "Ikshana" [3] is a wearable device that integrates Tesseract OCR and Google TTS engine.
- "Sehmoozer" [4] enables auto image/text capture by incorporating the edge detection and image capture with TTS capabilities.
- Natural Reader [6] is a TTS application with high-quality human-like voices with support for multiple languages and accents. It is also used in educational settings to enhance pronunciation and speaking skills.
- "Vivoice system [7]" is a reading assistance that integrates Tesseract OCR and eSpeak TTS. It uses Raspberry Pi to provide high accuracy and portability.
- Edupuganti et al. [8] proposed AI-assistive reading using Google Vision API for text detection and Android TTS for converting text into voice. Their solution shows real-time image capture via smartphone cameras to read medicine labels with high accuracy.
- Kumari et al. [9] showed application of YOLO for real-time object and text identification. They also convert visual data into descriptive audio feedback.
- Tan et al. [10] proposed "NaturalSpeech," which is an advanced TTS system that leverages a memory-enhanced variational autoencoder for end-to-end text-to-waveform generation.
- Kumar et al. [11] explored advanced techniques for handwriting recognition of Hindi characters.

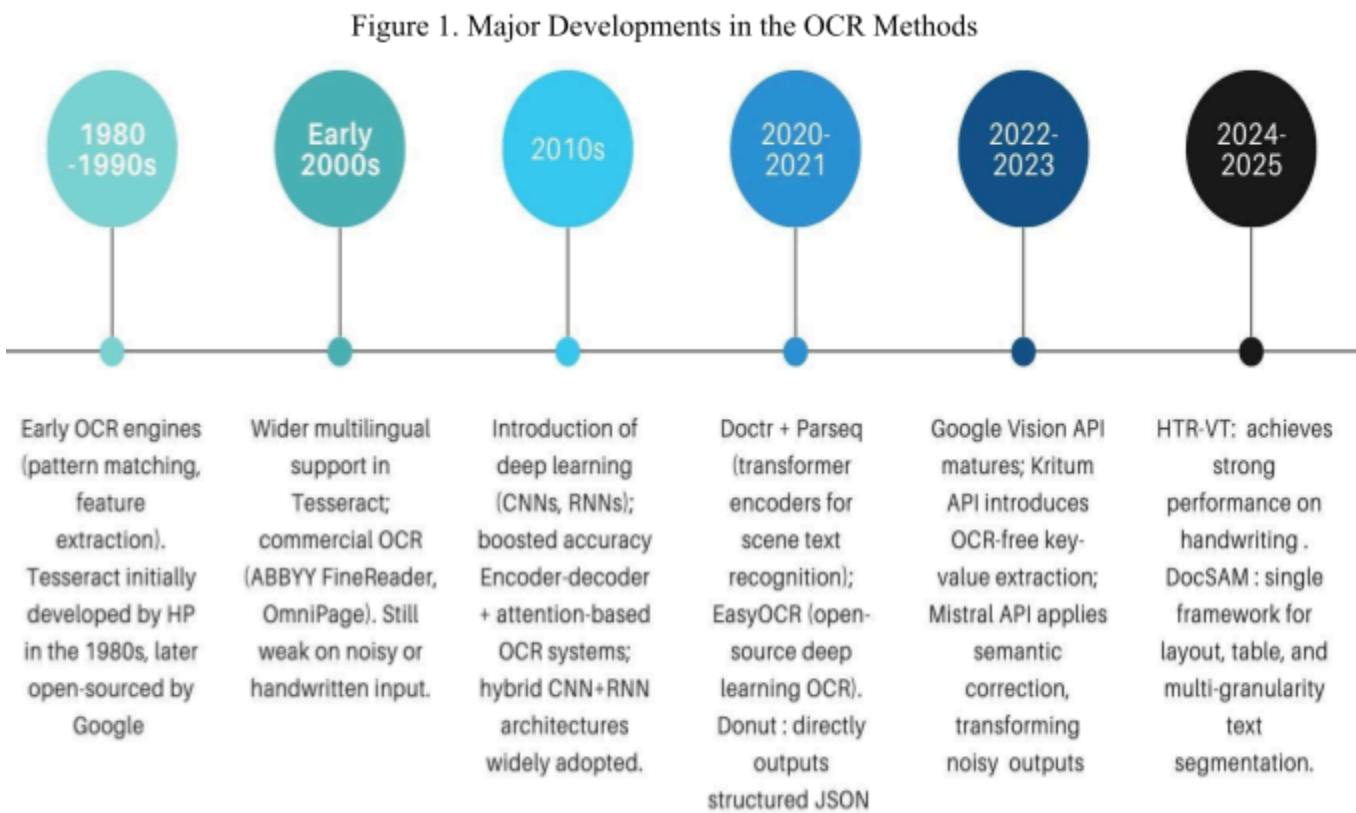


Table 1: Overview of OCR and OCR-Free Tools/API

Tool / API	OCR-Free	Multilingual Support	Offline Use	Structured Output	Post-OCR Correction
Doctr + Parseq	✗	✓	✓	✓	✗
Tesseract OCR (V4+)	✗	✓	✓	✗	✗
Google Vision API	✗	✓	✗	✓	✗
Mistral API	✗	✓	✗	✓	✓
EasyOCR	✗	✓	✓	✗	✗
Donut	✓	✓	✓	✓	✗

Table 2: Overview of OCR and OCR-Free Tools/APIs

Tool / API	Type	Key Feature
Doctr + Parseq	Deep Learning OCR	Layout-aware detection and recognition using transformer encoders; supports multilingual structured documents
Kritum API	OCR-free API	Structured key-value output without character-level OCR (JSON-based)
Tesseract OCR (v4+)	LSTM-based OCR	Open-source OCR engine with multilingual support; works offline but limited on noisy/complex layouts
Google Vision API	Cloud OCR API	Accurate OCR with multilingual and layout (bounding box) support; robust for noisy and handwritten text
Mistral API	LLM Post-OCR API	Semantic correction and structuring of OCR outputs; improves accuracy on noisy or historical text
EasyOCR	Lightweight Deep Learning OCR	Fast, open-source OCR with multilingual support; easy offline integration, supports >80 languages
Donut	OCR-free Transformer	End-to-end document understanding; directly outputs structured JSON from images, supports forms & receipts

4. Research Gaps and New Ideas

4.1 Gap 1: Weak syllabus alignment

Most assessment platforms focus mainly on grading answers, but they often neglect to check whether the questions themselves are properly tied to the course syllabus. This can result in students being asked about topics that were never formally covered, raising fairness concerns and reducing trust in the evaluation process.

Our approach: We make syllabus alignment a strict requirement. Every question generated is mapped to a specific, instructor-approved learning outcome (LO). To prevent redundancy, near-duplicate prompts are filtered out, so students are presented with varied but always relevant questions. This ensures both fairness and consistency.

4.2 Gap 2: Limited feedback beyond raw scores

In many current tools, feedback stops at a number or a letter grade. Students rarely see why they received that score, what parts of their response worked well, or what needs improvement. This lack of explanation limits learning value and can be discouraging.

Our approach: Each scoring dimension—such as accuracy, clarity, and completeness—comes with a short written explanation. Instructors can also revise scores or add their own comments. This way, students receive feedback they can actually act on, while departments benefit from a transparent record that supports reviews, audits, or appeals.

4.3 Gap 3: Narrow language coverage

In existing assessment tools launch with support for English only, and expansion to other languages is mostly not there. In a multilingual context like India, where many students move between English and Hindi, such a design quickly becomes a barrier.

Our approach: From the very first year, the system will support typed input in both English and Hindi, allowing students to participate in the language they are most comfortable with. This removes language as a barrier to engagement. Once the text-based setup has been tested and shown to work reliably, speech input will be added in later phases—broadening accessibility while keeping the system stable and dependable.

5. Research Questions for the project

The project is shaped around a set of focused research questions. These are designed to capture the core challenges of introducing AI into viva-style assessments. They look beyond technical accuracy to also consider fairness, ease of use, and academic rigor.

- **RQ1:** Can the system consistently ensure that every question stays within the approved syllabus, with a clear link to specific learning outcomes, while also filtering out near-duplicate prompts?
- **RQ2:** How well do the rubric scores generated by the AI correspond to faculty judgements in pilot studies, and does calibration improve this alignment over time?
- **RQ3:** Is the system responsive enough for everyday classroom use—producing questions within seconds and generating feedback reports in under half a minute?
- **RQ4:** Do students feel that the evaluation process is fair and helpful, and do instructors find that it reduces routine workload while still supporting deep learning?
- **RQ5:** Are the audit records—such as transcripts, rubric versions, and scoring notes—sufficiently detailed to make reviews, rechecks, and appeals straightforward?

6. Objectives

- **Syllabus alignment.** Ensure that every viva question is tied to an approved learning outcome, while filtering out duplicates so the assessment remains both fair and diverse.
- **Rubric-based scoring.** Apply an analytic rubric that assesses correctness, clarity, and completeness, with each score supported by a short explanation. Reports should be produced at both the individual student level and the class level.
- **Classroom responsiveness.** Make sure that questions are generated and reports are produced quickly enough to blend smoothly into live teaching sessions, without interrupting the natural flow of instruction.
- **Fairness and value.** Provide students with clear reasons for their marks, while giving instructors back time that would otherwise be spent on repetitive assessment tasks.
- **Transparency and auditability.** Maintain full records of transcripts, rubric details, and learning outcome mappings so that any decision can be checked, moderated, or appealed.
- **Invigilation support.** Design system with Invigilation support so that good academic practices can be practiced.

6. Work Flow

Fig. Detailed workflow for first year

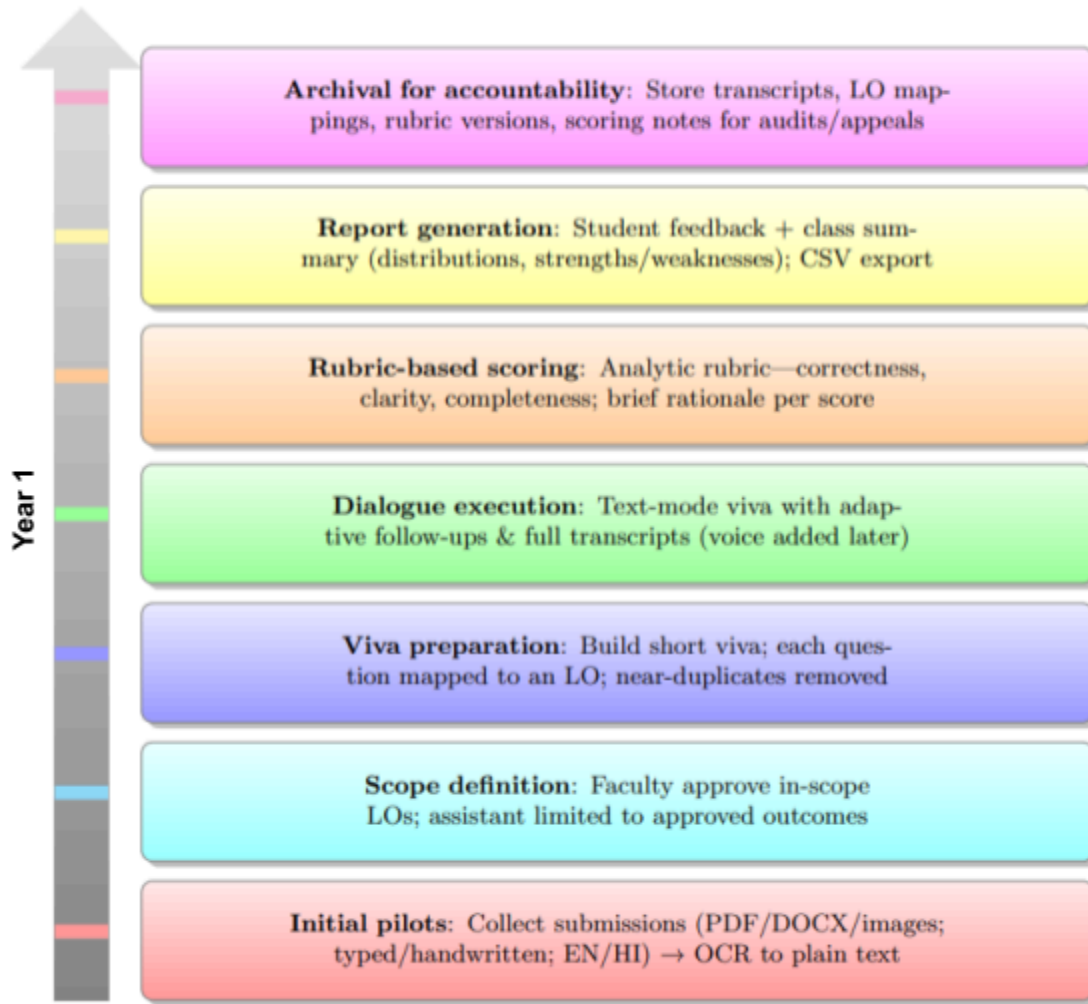
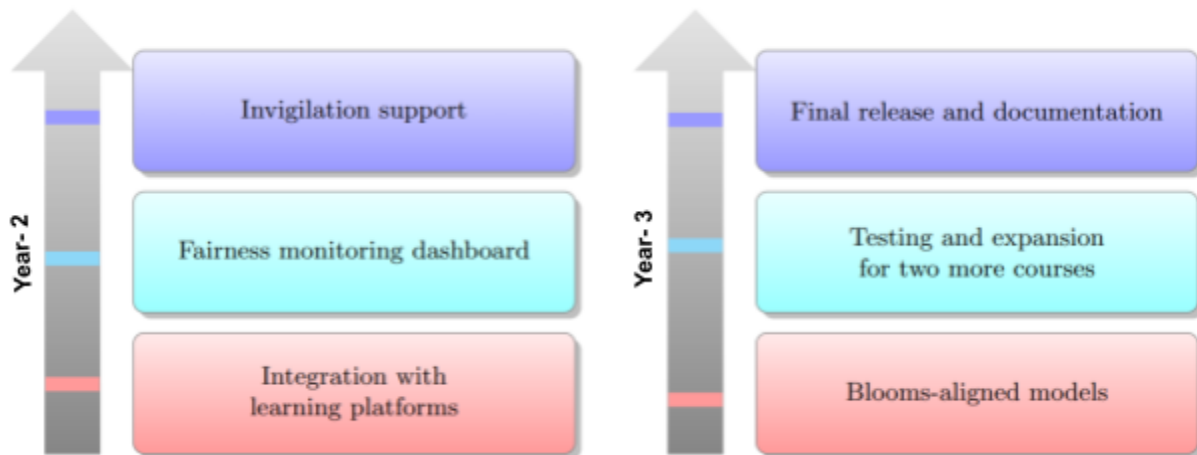


Fig. Workflow for 2nd and 3rd year



The AI-assisted viva system is planned to evolve gradually rather than appear as a finished product from the start. The following steps describe how the workflow will be done in Year 1 and then gradually expanded through Years 2 and 3.

1. **Initial pilots in two AI courses (Year-1).** Students will submit typed or handwritten work (in English or Hindi) as PDF, DOCX, or image files. Optical Character Recognition (OCR) is applied to standardize the input and convert it to plain text.
2. **Scope definition (Year-1).** Faculty members define and approve the learning outcomes (LOs) that are in scope. The assistant is constrained so that all questions are generated only from these approved outcomes.
3. **Viva preparation (Year-1).** Using the approved outcomes, develop **AI model** to conduct a short viva. Each question is linked to a specific LO, while near-duplicate prompts are automatically filtered out.
4. **Dialogue execution (Year-1 to Year-2).** In the first year, viva runs in text mode with adaptive follow-up questions and full transcript logging. In the second year, voice interaction (with text fallback) is added to make the exchange more natural. Both are basically **AI models** to conduct the viva in lab environment.
5. **Rubric-based scoring (Year-1).** Student responses are graded using an AI based rubric covering correctness, clarity, and completeness. Each score is accompanied by a short (one–two line) justification.
6. **Report generation (Year-1).** The **AI model** for reports: individual student evaluation report and a class-level summary showing distributions, common strengths, weaknesses in a exportable CSV option.
7. **Archival for accountability (Year-1).** All transcripts, learning outcome (LO) mappings, rubric versions, and scoring notes are stored securely. This creates a transparent record that can be revisited for rechecks or appeals.
8. **Integration with learning platforms (Year-2).** Connections to LMS/SSO systems like Moodle are introduced to keep rosters and grades in sync. For programming assignments, a plagiarism detection and similarity checks are also added.
9. **Fairness monitoring (Year-2).** Dashboards highlight group-level score differences, apply alert thresholds, and provide faculty with clear indicators to review and address any potential bias.
10. **Student activity and alertness (Year-2).** An optional AI based invigilation module to capture engagement signals during viva sessions. Invigilation summaries are provided to instructors on a dashboard.
11. **Bloom’s-aligned models (Year-3).** Dedicated models are designed to generate questions that target Bloom’s Level-4 skills, such as analysis, comparison, organization, and attribution.
12. **Testing and expansion (Year-3).** After controlled lab testing in two AI courses, the system is extended to additional courses, for example *Classical AI* and *Data Science*.
13. **Final release and documentation (Year-3).** The project archives a stable system that supports four AI-related courses. The final outputs will include clear documentation, tested evaluation scripts, training materials for instructors and students, and a detailed technical report or publication.

7. Time Schedule (3-Year Milestones)

The project is planned over three years, divided into distinct phases. Each phase has well-defined goals, checkpoints, and measurable outputs to track progress and maintain accountability. The following tables outline these details.

Table 1: Year 1 — Build and Pilot a Text-mode Viva System

	Focus	Key Activities	Deliverables / Metrics
Q1	Team and setup	Recruit & onboard 2 JRFs; Finalize pilot courses (Powers of AI, AI for Programmers); Draft LO matrix and scope policy; Procure & install multi-GPU workstation	Team onboarded; Workstation running; LO matrix (v1) approved
Q2	AI model for Text pipeline & OCR	Ingest typed/hand-written submissions (PDF/DOCX/images); OCR & text normalization (English/Hindi); Instructor console to approve 'in-scope' LOs	Text pipeline working end-to-end
Q3	AI model for Viva loop & scoring	Adaptive follow-ups with full transcript saved; Block near-duplicate questions; Analytic rubric scoring with rationales; Student report & class CSV export	End-to-end viva demo; Rubric outputs with rationales; Dashboards & CSV exports
Q4	Pilot and evaluation	Run pilots in both courses; Measure QWK, on-scope%, and latency; Privacy/security review and fixes; Prepare expansion recommendations	A system working on PDFs with high QWK

Table 2: Year 2 — Scale and Integrate Advanced Features

	Focus	Key Activities	Deliverables / Metrics
Q1	AI model for Voice Viva	Add voice input with fallback to text; Map ASR transcripts to approved LOs; Check accuracy and latency	Voice viva prototype running; Baselines: WER & response time
Q2	LMS/SSO & programming support	Connect to LMS (e.g., Moodle); Enable institute SSO; Add code sandbox & basic similarity checks	SSO sign-in working; Gradebook sync demo; Sandbox demo
Q3	Fairness & transparency checks	Collect pilot data for group fairness; Build dashboard showing score distributions (anonymized); Define thresholds for alerts; Write faculty guide on bias	Fairness dashboard; Policy note on alerts
Q4	AI based Invigilation Support for student activities	Develop module to detect activity/alertness via webcam (lightweight & privacy-preserving); Pilot with opt-in groups; Ethics review	Prototype alertness dashboard; Pilot data (opt-in); Privacy/ethics note

Table 3: Year 3 — Bloom's-Aligned and Multi-Course Models

	Focus	Key Activities	Deliverables / Metrics
Q1	AI based Bloom's Taxonomy-aligned question model	Design model aligned with Bloom's Level-4 outcomes; Create training datasets from AI course materials; Validate mapping with faculty	Prototype Bloom's-aligned generator; LO mapping sheet; Faculty review report

Q2	Lab testing on pilot AI courses	Deploy model in lab environment; Test on Powers of AI & AI for Programmers; Collect metrics	Lab test reports; Evaluation dashboard; Calibration notes
Q3	AI models training extend to two more AI courses	Train system for Classical AI & Data Science; Small-scale pilots (opt-in); Refine rubrics & question pool	Models for 4 AI courses; Pilot feedback reports; Expanded LO/question bank
Q4	Final documentation & release	Compile workflows, data practices, evaluation scripts; Release version for 4 AI courses; Prepare teaching kit; Draft paper	Documentation bundle; Release package; Instructor kit; Final report/paper draft

8. Deliverables

Year 1 - Build and Pilot (Text Mode)

- Recruitment of the core team is completed, the workstation is installed, and the first version of the learning outcome (LO) matrix is formally approved.
- A text-based pipeline is demonstrated end-to-end, including audit logging and a pre-check to confirm that all questions stay within scope.
- A complete viva prototype is prepared, showing rubric-based outputs with short explanations, along with dashboards and CSV export functions.
- Pilot runs are conducted, with results reported on key metrics such as Quadratic Weighted Kappa (QWK) agreement, percentage of on-scope questions, and latency. A written pilot report is produced.

Year 2 - Extension and Integration

- A voice-enabled viva prototype is introduced, and baseline values for word error rate (WER) and system response times are established.
- Single sign-on (SSO) with roster/gradebook synchronization is tested in a demo environment, and a code sandbox prototype is added for programming-related courses.
- An early version of a fairness dashboard is made available, along with a draft policy on thresholds and alerts.
- A prototype dashboard is piloted to show student activity and alertness data (invigilation support). Pilot reports include data on usage, along with a privacy and ethics disclaimer.

Year 3 - Final System and Bloom's-aligned Models

- A prototype question generator aligned to Bloom's Level-4 outcomes (analysis, comparison, organization, attribution) is completed, with validation against the LO matrix and a faculty review report.
- Laboratory testing is carried out for two AI pilot courses, with results summarized in an evaluation dashboard and calibration updates.
- The system is extended to cover four AI-related courses, with an expanded LO/question bank and pilot feedback reports.
- Final deliverables include a documentation set, a release package covering all four courses, instructor training materials, and a consolidated final report with a draft publication.

9. Team & Roles

Project Lead (Faculty at IIT BHU)

- Ensure the project stays aligned with the syllabus and meets all defined learning outcomes (LOs).
- Approve key milestones such as pilot readiness, publications, and rubrics.
- Lead review meetings to discuss scope or policy issues and resolve problems as they arise.

JRFs (Data, Models, and Integration)

- Develop and test models for generating questions and evaluating their quality.
- Prepare student- and class-level reports in both Hindi and English.
- Integrate the system with LMS/SSO platforms, build invigilation models.
- Run accuracy and performance tests.
- Maintain updated scripts, tools, and processes to ensure reproducibility.

Industry Mentor (SARAS)

- Provide guidance on system design and scalability to ensure long-term adaptability.
- Support both back-end and front-end development, as well as LMS/SSO integration.
- Advise on best practices for documentation, design choices, and usability.
- Test and validate pilot deployments to ensure they are reliable and effective.

Committee for Evaluation and Rubrics

- Ensure rubrics and learning outcomes remain clear, fair, and consistent.
- Review proposed changes collectively and approve them as a group.
- Escalate unresolved issues to the academic lead for final decision.

Student Assistants (UG/PG TAs)

- Assist in preparing sample data, setting up sessions, and running pilot tests.
- Collect and identify rubric examples, including unique or edge cases.
- Keep project notes and documentation concise and well-organized.

Institute Administrative Support

- Provide support for the initial AI courses (AI for Programmers and Powers of AI), with the option to expand to additional courses later.
- Allocate lab space and arrange workstation setups for viva sessions.

References

1. Velmurugan, Sathya Narayanan, K. Azarudeen, Selvalakshmi Sekar, and Bharath Singh Jebaraj. "A Next-Gen Automated Reading Companion for the Visually Impaired." In 2023 International Conference on Research Methodologies in Knowledge Management, Artificial Intelligence and Telecommunication Engineering (RMKMATE), pp. 1-7. IEEE, 2023.
2. Mahir, M. A. M., M. N. M. Hussain, R. D. D. Perera, Y. A. M. Upendra, C. J. Wickramaratne, and Dharshana Kasthurirathna. "SPAVIS: mobile application for visually impaired based on assistive software and volunteerism." In 2022 4th International Conference on Advancements in Computing (ICAC), pp. 310-315. IEEE, 2022.
3. Chauhan, Shreya, Deeksha Patkar, Avikshit Dabholkar, and Kautuki Nirgun. "Ikshana: Intelligent Assisting system for visually challenged people." In 2021 2nd International Conference on Smart Electronics and

- Communication (ICOSEC), pp. 1154-1160. IEEE, 2021.
4. Gamage, N. D. U., K. W. C. Jayadewa, and J. A. D. C. A. Jayakody. "Document reader for vision impaired elementary school children to identify printed images." In 2019 International Conference on Advancements in Computing (ICAC), pp. 279-284. IEEE, 2019.
 5. Wongta, Pitchakorn, and Thanarat H. Chalidabhongse. "Vision-based bus route number reader for visually impaired travelers." In 2018 2nd International Conference on Imaging, Signal Processing and Communication (ICISPC), pp. 64-69. IEEE, 2018.
 6. Fitria, Tira Nur. "Utilizing Text-to-Speech Technology: Natural Reader in Teaching Pronunciation." JETLEE: Journal of English Language Teaching, Linguistics, and Literature 2, no. 2 (2022): 70-78.
 7. Prabha, R., M. Razmah, G. Saritha, R. M. Asha, and R. Gayathiri. "Vivoice-Reading Assistant for the Blind using OCR and TTS." In 2022 International Conference on Computer Communication and Informatics (ICCCI), pp. 01-07. IEEE, 2022.
 8. Edupuganti, Sai Aishwarya, Vijaya Durga Koganti, Cheekati Sri Lakshmi, Ravuri Naveen Kumar, and Ramya Paruchuri. "Text and Speech Recognition for Visually Impaired People using Google Vision." In 2021 2nd International Conference on Smart Electronics and Communication (ICOSEC), pp. 1325-1330. IEEE, 2021.
 9. Kumari, C. Shyamala, S. Durai, Dornadula Sudha Karthisri, Maliga Hemanth Kumar, and Potluru Venkata Sai Sukhesh. "Enhancing Accessibility for the Visually Impaired through Text Voice-Activated Images." In 2024 2nd International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS), pp. 804-812. IEEE, 2024.
 10. Tan, Xu, Jiawei Chen, Haohe Liu, Jian Cong, Chen Zhang, Yanqing Liu, Xi Wang et al. "Naturalspeech: End-to-end text-to-speech synthesis with human-level quality." IEEE Transactions on Pattern Analysis and Machine Intelligence, 2024.
 11. Kumar, Ankit, Surbhi Bhatia, Mohammad R. Khosravi, Arwa Mashat, and Parul Agarwal. "Semantic and context understanding for sentiment analysis in Hindi handwritten character recognition using a multiresolution technique." ACM Transactions on Asian and Low-Resource Language Information Processing 23, no. 1 (2024): 1-22.
 12. Baek, J., Kim, G., Lee, J., Park, S., Han, D., Yun, S., ... & Lee, H. (2021). Parseq: A simple and strong baseline for scene text recognition with permutation-invariant training. *arXiv preprint arXiv:2107.10985*.
 13. Google Cloud. (2022). Cloud Vision API documentation. Retrieved from <https://cloud.google.com/vision>
 14. JaidedAI. (2020). EasyOCR: Ready-to-use OCR with 80+ supported languages. GitHub repository. Retrieved from <https://github.com/JaidedAI/EasyOCR>
 15. Kim, G., Park, S., Lee, J., & Lee, H. (2022). Donut: Document understanding transformer without OCR. *arXiv preprint arXiv:2111.15664*.
 16. Krutrim. (2023). Kritum API for key-value information extraction. Documentation. Retrieved from <https://krutrim.com>
 17. Li, C., Chen, X., Tang, Y., & Shen, C. (2024). HTR-VT: Handwritten text recognition with vision transformer. *arXiv preprint arXiv:2409.08573*.
 18. Li, Y., Yin, X., & Liu, C. (2025). DocSAM: Unified document image segmentation via query decomposition and heterogeneous mixed learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2025)*.
 19. Mistral AI. (2023). Mistral 7B model: Technical report. Retrieved from <https://mistral.ai>
 20. Smith, R. (2007). An overview of the Tesseract OCR engine. *Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, 629–633. IEEE.